

# 1. Array Creation Functions

```
In [1]: import numpy as np
```

```
In [2]: a = np.array([1, 2, 3])  
print("Array a:", a)
```

Array a: [1 2 3]

```
In [3]: b = np.arange(0, 10, 2) # Values from 0 to 10 with step 2  
print("Array b:", b)
```

Array b: [0 2 4 6 8]

```
In [4]: c = np.linspace(0, 1, 5) # 5 values evenly spaced between 0 and 1  
print("Array c:", c)
```

Array c: [0. 0.25 0.5 0.75 1. ]

```
In [5]: d = np.zeros((2, 3)) # 2x3 array of zeros  
print("Array d:\n", d)
```

Array d:  
[[0. 0. 0.]  
 [0. 0. 0.]]

```
In [6]: e = np.ones((3, 2)) # 3x2 array of ones  
print("Array e:\n", e)
```

Array e:  
[[1. 1.]  
 [1. 1.]  
 [1. 1.]]

```
In [7]: # Create an identity matrix  
f = np.eye(4) # 4x4 identity matrix  
print("Identity matrix f:\n", f)
```

Identity matrix f:  
[[1. 0. 0. 0.]  
 [0. 1. 0. 0.]  
 [0. 0. 1. 0.]  
 [0. 0. 0. 1.]]

# 2. Array Manipulation Functions

```
In [8]: # Reshape an array  
a1 = np.array([1, 2, 3])  
reshaped = np.reshape(a1, (1, 3)) # Reshape to 1x3  
print("Reshaped array:", reshaped)
```

Reshaped array: [[1 2 3]]

```
In [9]: # Flatten an array  
f1 = np.array([[1, 2], [3, 4]])
```

```
flattened = np.ravel(f1) # Flatten to 1D array
print("Flattened array:", flattened)
```

Flattened array: [1 2 3 4]

```
In [10]: # Transpose an array
e1 = np.array([[1, 2], [3, 4]])
transposed = np.transpose(e1) # Transpose the array
print("Transposed array:\n", transposed)
```

Transposed array:

```
[[1 3]
 [2 4]]
```

```
In [11]: # Stack arrays vertically
a2 = np.array([1, 2])
b2 = np.array([3, 4])
stacked = np.vstack([a2, b2]) # Stack a and b vertically
print("Stacked arrays:\n", stacked)
```

Stacked arrays:

```
[[1 2]
 [3 4]]
```

### 3. Mathematical Functions

```
In [12]: # add 2 to each element in array
g=np.array([1,2,3,4,5,])
added=np.add(g,2)
print('added 2 to g:',added)
```

added 2 to g: [3 4 5 6 7]

```
In [13]: # square each element
squared=np.power(g,2)
print('2 squared to g:',squared)
```

2 squared to g: [ 1 4 9 16 25]

```
In [14]: # square root of each element
sqrt_g=np.sqrt(g)
print('sqrt_g:',sqrt_g)
```

sqrt\_g: [1. 1.41421356 1.73205081 2. 2.23606798]

```
In [15]: print(a1)
print(g)
```

```
[1 2 3]
[1 2 3 4 5]
```

```
In [16]: # Dot product of two arrays
dot_product=np.dot(a1,g)
print('dot_product of a1 and g:',dot_product)
```

```
-----
ValueError                                Traceback (most recent call last)
Cell In[16], line 2
      1 # Dot product of two arrays
----> 2 dot_product=np.dot(a1,g)
      3 print('dot_product of a1 and g:',dot_product)

ValueError: shapes (3,) and (5,) not aligned: 3 (dim 0) != 5 (dim 0)
```

```
In [17]: dot_product=np.dot(a1,a) # a=[1,2,3] same no. of element
print('dot_product of a1 and g:',dot_product)
```

dot\_product of a1 and g: 14

## 4. Statistical Functions

```
In [18]: s = np.array([1, 2, 3, 4])
mean = np.mean(s)
print("Mean of s:", mean)
```

Mean of s: 2.5

```
In [20]: # Standard deviation of an array,min=1 & max = 4
std_dev = np.std(s)
print("Standard deviation of s:", std_dev)
```

Standard deviation of s: 1.118033988749895

## 5. Linear Algebra Functions

```
In [21]: # Create a matrix
matrix = np.array([[1, 2], [3, 4]])
```

```
In [22]: matrix
```

```
Out[22]: array([[1, 2],
               [3, 4]])
```

```
In [30]: # Determinant of a matrix = 4*1-2*3== -2
determinant = np.linalg.det(matrix) # matrix should be a square like -(2,2)(3,3,
print('Determinant of matrix:',determinant)
```

Determinant of matrix: -2.0000000000000004

```
In [31]: # Inverse of a matrix
inverse = np.linalg.inv(matrix)
print("Inverse of matrix:\n", inverse)
```

Inverse of matrix:  
[[-2. 1.]  
 [ 1.5 -0.5]]

## 6. Random Sampling Functions

```
In [32]: random_vals = np.random.rand(3) # Array of 3 random values between 0 and 1
print("Random values:", random_vals)
```

Random values: [0.49688363 0.14815013 0.54324399]

```
In [38]: # Generate random integers
np.random.seed(0) # seed means values dont change

rand_ints = np.random.randint(0, 10, size=5) # Random integers between 0 and 10
print("Random integers:", rand_ints)
```

Random integers: [5 0 3 3 7]

## 7. Boolean & Logical Functions

```
In [42]: ltest=np.array([True,False,True])
alltrue=np.all(ltest)
print('all elements are true:',alltrue)
```

all elements are true: False

```
In [45]: ltest=np.array([True,False,True])
anytrue=np.any(ltest)
print('any elements are true:',anytrue)
```

any elements are true: True

## 8. Set Operations

```
In [46]: # Intersection of two arrays common element
set_a = np.array([1, 2, 3, 4,5])
set_b = np.array([3, 4, 5, 6,7])
intersection = np.intersect1d(set_a, set_b)
print("Intersection of a and b:", intersection)
```

Intersection of a and b: [3 4 5]

```
In [47]: # Union of two arrays remove duplicate and combine all
union = np.union1d(set_a, set_b)
print("Union of a and b:", union)
```

Union of a and b: [1 2 3 4 5 6 7]

## 9. Array Attribute Functions

```
In [48]: # Array attributes
r = np.array([1, 2, 3])
shape = r.shape # Shape of the array
size = r.size # Number of elements
dimensions = r.ndim # Number of dimensions
dtype = r.dtype # Data type of the array

print("Shape of r:", shape)
print("Size of r:", size)
```

```
print("Number of dimensions of r:", dimensions)  
print("Data type of r:", dtype)
```

Shape of r: (3,)  
Size of r: 3  
Number of dimensions of r: 1  
Data type of r: int64

## 10. Other Functions

```
In [49]: # copy of array  
cp=np.array([1,2,3])  
cp1=np.copy(cp)  
cp1
```

Out[49]: array([1, 2, 3])

```
In [50]: cp.nbytes # size in bytes
```

Out[50]: 24

```
In [51]: shared_memory=np.shares_memory(cp,cp1) # false means memory not shared  
shared_memory
```

Out[51]: False

```
In [ ]:
```